MINI PROJECT REPORT

on

MICROCONTROLLER AI LAB

An AI-Powered Interactive Electronics Learning Platform

with Circuit Validation, Code Generation and In-Browser Simulation

Supported boards: Arduino Uno R3 and ESP32 DevKit V1

Submitted by

NAME: SANTHOSH P
REG NO: 911723104092

Programme: Full Stack AI BootCamp

Project: Microcontroller and AI Laboratory

Academic Year: 2025 – 2026

INDEX

1. Abstract 3
2. Introduction 4
3. Aim and Technology Stack 5
4. System Architecture 7
5. Work Flow 9
6. Project Structure 11
7. Important Code Concepts 13
8. REST API Endpoints 19
9. Project Screenshots 20
10. Result and Discussion 24
11. Advantages, Limitations and Future Scope 25
12. Conclusion 26
13. References 27

1. Abstract

Learning microcontrollers is difficult for beginners. A wrong wire, a missing current-limiting resistor or a 5 V signal fed into a 3.3 V pin can damage a board, and compiler or wiring errors are hard to understand without a mentor. Video tutorials are passive and professional circuit tools have a steep learning curve.

Microcontroller AI Lab is a full-stack web application that acts as an interactive electronics tutor. A student selects a board (Arduino Uno R3 or ESP32 DevKit V1) and a lesson (LED Blinking, Push Button or DHT11 Sensor), drags components onto an SVG workspace, and wires the pins. The circuit is then checked by a deterministic rule-based validator that builds an electrical netlist using a Disjoint Set Union (union-find) structure. Board-compatible Arduino-style code is generated from the pins that were actually wired, a lightweight in-browser simulator animates a virtual LED, button and DHT11 sensor, and an AI tutor explains concepts, generates code and troubleshoots problems.

The central design principle is separation of responsibility: the validator and the board registry decide what is correct, while the AI (Anthropic Claude, called only through the Node.js/Express backend) explains and assists but can never override a validation result. API keys stay on the server, the backend re-validates every circuit it receives, and a usage dashboard shows token consumption and cost. The application was deployed online and tested with all three lessons on both supported boards.

Keywords: Microcontroller, Arduino Uno, ESP32, Circuit Validation, Union-Find, Generative AI, Node.js, Express, SVG, Simulation, Education.

2. Introduction

2.1 Background

Embedded systems and the Internet of Things rely on microcontroller boards such as the Arduino Uno and the ESP32. Students must understand pin capabilities, logic voltages, resistors, polarity and code at the same time. Physical labs are limited by equipment and by the risk of damaging parts, so a safe virtual environment with immediate feedback is valuable.

Large language models can explain electronics in plain language, but they sometimes invent pins or declare a faulty circuit correct. A learning platform must therefore combine the explanatory power of AI with deterministic, verifiable rules.

2.2 Problem Statement

Beginners need a safe, board-aware environment in which they can build a circuit visually, learn why it is right or wrong, obtain matching code, see the behaviour simulated, and ask questions – without risking hardware and without being misled by AI hallucinations.

2.3 Objectives

- Provide a board selector driven by a central registry so that new boards can be added without rewriting the application.
- Provide a drag-and-drop SVG circuit workspace with pin-to-pin wiring.
- Validate circuits with deterministic, board-specific rules that report clear errors and warnings.
- Generate Arduino-style code that uses the real pins resolved from the student's circuit.
- Simulate supported behaviour (LED, push button, DHT11) in the browser and label it clearly as simulated.
- Integrate an AI tutor through the backend only, with guardrails so the AI never overrides validation.
- Track AI token usage, cost and remaining credit on a dashboard.

2.4 Scope

The prototype offers three lessons for two boards. It is a learning tool, not a compiler, an electrical simulator or a firmware emulator; physical hardware results may differ. Circuits live in browser memory and no database is required.

3. Aim and Technology Stack

3.1 Aim

To design and implement a full-stack, AI-powered web platform that teaches microcontroller basics through interactive circuit building, deterministic validation, board-compatible code generation, in-browser simulation and an AI tutor, for the Arduino Uno R3 and the ESP32 DevKit V1.

3.2 Technology Stack

Layer	Technology	Purpose
Frontend	HTML5, CSS3, Vanilla JavaScript	User interface, no framework or bundler
Graphics	Inline SVG	Boards, components, pins, wires, LED glow
Backend	Node.js 18+, Express 4	REST API, static file serving
Configuration	dotenv	Secrets and settings in backend/.env
AI	Anthropic Claude (Messages API)	Explain, generate code, troubleshoot
Usage analytics	Provider Usage / Cost API + local log	Token, cost and balance dashboard
Storage	JSON file (backend/data/usage.json)	Application-tracked usage only
Version control	Git	Source management
Hosting	Render (web service)	Online deployment of the prototype

3.3 Supported Boards

Feature	Arduino Uno R3	ESP32 DevKit V1
Microcontroller	ATmega328P (AVR)	ESP32-WROOM-32 (dual-core)
Logic voltage	5 V	3.3 V
Digital pins	D0 – D13	GPIO 2, 4, 5, 12–19, 21–23, 25–27, 32, 33
Analog inputs	A0 – A5	ADC1 pins (e.g. 32–36, 39)
PWM pins	D3, D5, D6, D9, D10, D11	All output-capable GPIOs
Power pins	5V, 3V3	3V3, VIN
Restrictions	D0/D1 serial, D13 on-board LED	GPIO 34–39 input-only; GPIO 2, 12, 15 boot-strapping

3.4 Lessons

Lesson	What the student builds	Simulator support
LED Blinking	GPIO → resistor → LED anode, cathode → GND	pinMode, digitalWrite, delay
Push Button	Button to an input pin and GND (INPUT_PULLUP) controlling an LED	plus digitalRead
DHT11 Sensor	VCC, GND and DATA of a DHT11 wired to the board	Simulated temperature / humidity

4. System Architecture

The system follows a client–server architecture. The browser contains the user interface, the SVG workspace, the validator, the simulator and the code generator. The Express backend serves the frontend, exposes REST endpoints, re-validates circuits and is the only component that communicates with the AI provider.

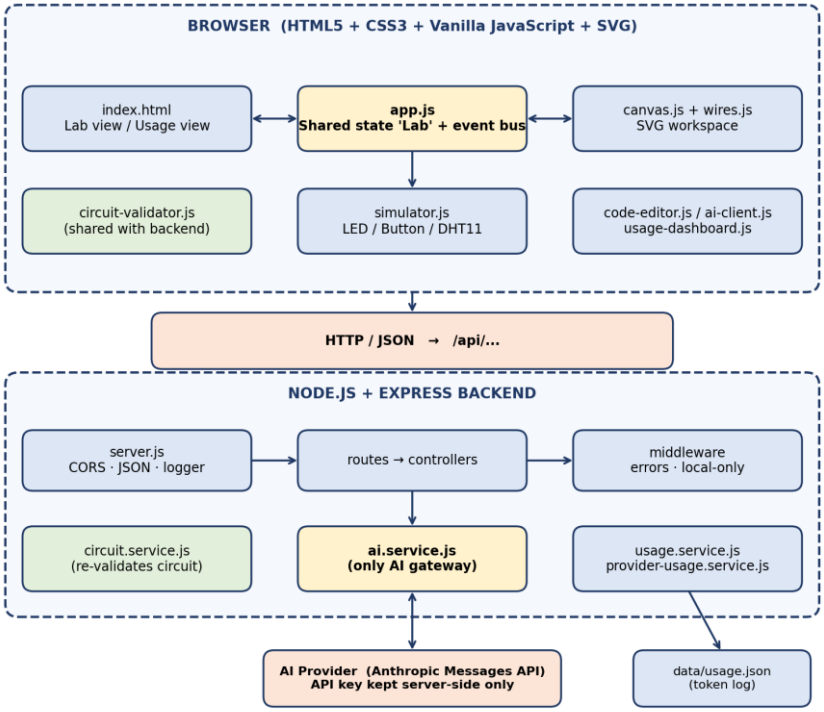


Fig 4.1: Architecture of Microcontroller AI Lab

4.1 Separation of Responsibility

Question	Decided by	Never decided by
Is the circuit valid?	Deterministic validator	The AI
Which pins and voltages does a board have?	Board registry	The AI or the browser
Explanation, tutoring, troubleshooting	AI tutor (Claude)	–
Does the code work on hardware?	Not claimed by the app	The AI or the simulator

4.2 Key Design Decisions

- **Registry-driven boards:** pins, capabilities, restrictions and default pins are data in board-registry.js, not hard-coded in lessons.

- **Shared modules:** board-registry, components, lessons and circuit-validator are written as UMD modules and used by both the browser and the Node backend, so client and server can never disagree.
- **Zero-trust server:** the backend rebuilds board capabilities and re-runs validation before every AI call instead of trusting the client.
- **API key isolation:** AI credentials exist only in backend/.env and are never sent to the browser.
- **No eval():** the simulator runs a structured model of the lesson; it never executes generated or AI-written code.

5. Work Flow

5.1 Student Workflow

1. Open the application and select a board (Arduino Uno R3 or ESP32) and a lesson from the navigation bar.
2. Drag the board, LED, resistor, push button or DHT11 from the component library into the SVG workspace (or click a part to add it).
3. Turn Wire Mode on and click two pins to create a wire; select a wire colour and delete or re-route wires as needed.
4. Click Validate Circuit. The validator lists errors and warnings and the badge changes to “Circuit valid” when no error remains.
5. Open the Code tab and click Generate Code to receive board-compatible code, line explanations and required libraries.
6. Open the Simulation tab and press Run to see the virtual LED, press the virtual button or move the DHT11 sliders; the serial output shows what happened.
7. Use the AI Tutor tab to ask questions, generate code or troubleshoot using the real circuit data.
8. Open AI API Key Usage to review tokens, cost and balance, then continue with the next lesson.

5.2 Flowchart

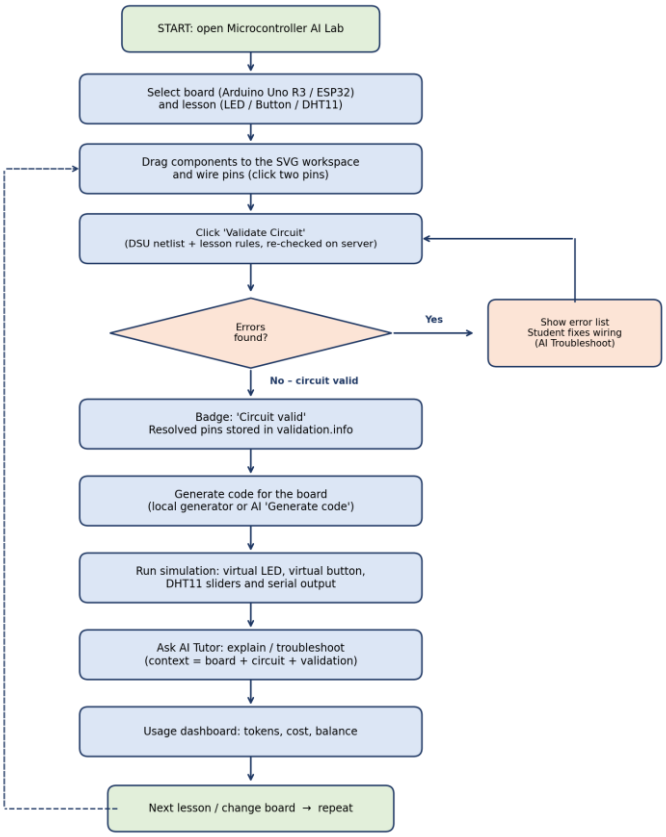


Fig 5.1: Operational flowchart of a lesson

5.3 Request Flow for an AI Question

1. ai-client.js collects the context: selected board, lesson, circuit, current code and last validation.
2. The browser sends POST /api/ai/explain (or generate-code / troubleshoot) to the backend.
3. The controller validates the request size and shape, then rebuilds the board capabilities from the server registry and re-runs validation.
4. ai.service.js adds the guardrail system prompt, calls the Anthropic Messages API with a timeout, and records the token usage.
5. The reply is returned to the browser and shown in the chat or the Code tab.

6. Project Structure

6.1 Folder Structure

```
microcontroller-ai-lab/
├── frontend/
│   ├── index.html                // layout: Lab view + Usage view
│   ├── css/styles.css            // theme and responsive layout
│   └── js/
│       ├── board-registry.js     // boards, pin capabilities   (shared)
│       ├── components.js         // component catalogue        (shared)
│       ├── lessons.js            // lesson definitions         (shared)
│       ├── circuit-validator.js  // deterministic validation   (shared)
│       ├── canvas.js             // SVG workspace, drag, zoom
│       ├── wires.js              // wire drawing / selection
│       ├── simulator.js          // LED / button / DHT11 simulation
│       ├── code-editor.js        // code generation + explanation
│       ├── ai-client.js          // AI tutor chat client
│       ├── usage-dashboard.js    // token usage dashboard UI
│       └── app.js                // orchestrator + shared state
├── backend/
│   ├── src/
│   │   ├── server.js             // Express entry point
│   │   ├── config/board-registry.js
│   │   ├── routes/      (boards, ai, circuit, usage, index)
│   │   ├── controllers/ (boards, ai, circuit, usage)
│   │   ├── services/    (ai, circuit, usage, provider-usage)
│   │   ├── middleware/  (errors, local-only)
│   │   └── utils/validate.js
│   ├── data/usage.json           // generated usage log
│   ├── .env.example              // configuration template
│   └── package.json
├── docs/PROJECT_DOCUMENTATION.md
├── proof/                        // application screenshots
└── README.md
```

6.2 Frontend Modules

File	Responsibility
app.js	Creates the shared state object Lab (board, lesson, components, connections), an event bus, helpers to add/remove components and wires, and connects all modules
board-registry.js	Single source of truth for boards: pins, PWM/analog capability, input-only pins, restrictions, power pins, default lesson pins
components.js / lessons.js	Component catalogue with pin positions and internal bridges; lesson descriptions, required components, concepts and steps
circuit-validator.js	Builds the netlist with union-find and applies board and lesson rules
canvas.js / wires.js	SVG rendering of boards and parts, dragging, zoom/pan, wire drawing, colours and route handles
code-editor.js	Generates code from resolved pins, explains lines, lists libraries and unsupported calls

File	Responsibility
simulator.js	Run / Stop / Reset, virtual LED, virtual button, DHT11 sliders, serial output
ai-client.js / usage-dashboard.js	AI tutor chat; usage cards, charts and tables

6.3 Backend Modules

File	Responsibility
server.js	Express setup: CORS, JSON body limit (256 kb), request logger (no bodies), static frontend, /api routes, error handlers
routes/*	Maps URLs to controllers (boards, ai, circuit, usage)
controllers/*	Validates requests, rebuilds the authoritative context, calls services
services/ai.service.js	The only AI gateway; prompt building, provider call, timeout, error mapping, usage recording
services/circuit.service.js	Normalises circuits and runs the shared validator against the backend registry
services/usage.service.js	Stores token metadata in usage.json (last 5000 records)
services/provider-usage.service.js	Reads provider-reported usage and cost for the dashboard
middleware/errors.js, local-only.js	Uniform JSON errors; restricts usage endpoints to localhost

6.4 Code Size

Part	Files	Approx. lines of code
Frontend JavaScript	11	4,900
Frontend HTML + CSS	2	1,460
Backend JavaScript	18	1,400

7. Important Code Concepts

This section explains the key programming ideas of the project. Snippets are shortened for readability and contain no credentials.

7.1 Central Board Registry

Every board is a plain object holding its logic voltage, digital, analog and PWM pins, power and ground pins and per-pin restrictions. buildPinInfo() expands these arrays into a per-pin lookup, including input-only pins such as GPIO34–39 on the ESP32. Validation, code generation, the pin tooltips and the AI context all read from this registry, so adding a board means adding one object.

```
'esp32-devkit': {
  id: 'esp32-devkit',
  name: 'ESP32 DevKit',
  platform: 'ESP32',
  mcu: 'ESP32-WROOM-32 (dual-core, Wi-Fi/BT)',
  logicVoltage: 3.3,
  digitalPins: ESP32_DIGITAL.slice(),
  analogPins:  ESP32_ANALOG.slice(),   // ADC1 only (ADC2 conflicts with Wi-Fi)
  pwmPins:     ESP32_PWM.slice(),
  powerPins: ['3V3', 'VIN'],
  groundPins: ['GND'],
  pinInfo: buildPinInfo(ESP32_DIGITAL, ESP32_ANALOG, ESP32_PWM,
                        ESP32_PIN_RESTRICTIONS, ESP32_INPUT_ONLY),
  defaultPins: { led: 'GPIO2', ... }
}
```

7.2 Shared UMD Modules (Browser + Node.js)

Four frontend files are wrapped in an immediately invoked function that exports through module.exports in Node.js and through a global variable in the browser. The backend simply require()s the same circuit-validator.js, so the client and the server always apply identical rules.

```
(function (global) {
  'use strict';
  var isNode = typeof module !== 'undefined' && module.exports;
  var BoardRegistry = isNode ? require('./board-registry') : global.BoardRegistry;
  // ... validator code ...
  if (isNode) module.exports = api;
  else global.CircuitValidator = api;
})(typeof window !== 'undefined' ? window : this);
```

7.3 Circuit Data Model

A circuit is described by components and connections. A pin reference has the form componentId.pinId, which keeps the model small, easy to serialise to the backend and easy to analyse.

```
const circuit = {
  board: 'arduino-uno-r3',
  lesson: 'led',
  components: [
    { id: 'board-1',    type: 'arduino-uno-r3', x: 80,  y: 140 },
    { id: 'led-1',      type: 'led',      x: 620, y: 180, properties: { color: 'red' }
},
    { id: 'resistor-1', type: 'resistor', x: 470, y: 210, properties: { resistance: 220
} }
  ],
  connections: [
    { id: 'wire-1', from: 'board-1.D8',      to: 'resistor-1.pin1' },
    { id: 'wire-2', from: 'resistor-1.pin2', to: 'led-1.anode' },
    { id: 'wire-3', from: 'led-1.cathode',   to: 'board-1.GND' }
  ]
};
```

7.4 Netlist Construction with Disjoint Set Union

To understand what is electrically connected, the validator treats every pin as a node and every wire as a union. Pins that end up in the same set share one electrical net. Path compression keeps find() fast. All GND pins of a board are merged into one net, while power rails stay separate because they carry different voltages. After this step, questions such as “is the LED cathode on ground?” become a simple comparison of two net identifiers.

```
DSU.prototype.find = function (x) {
  this.add(x);
  while (this.parent[x] !== x) {
    this.parent[x] = this.parent[this.parent[x]];   // path compression
    x = this.parent[x];
  }
  return x;
};
DSU.prototype.union = function (a, b) {
  var ra = this.find(a), rb = this.find(b);
  if (ra !== rb) this.parent[rb] = ra;
};

state.connections.forEach(function (c) {
  if (resolvePin(state, c.from) && resolvePin(state, c.to)) dsu.union(c.from, c.to);
});
```

7.5 Rule-Based Lesson Validation

After the structural checks (a board exists, wires use real pins, required components are present) the validator tries the accepted LED topologies: GPIO → resistor → LED anode with the cathode on GND (topology A), or GPIO → LED anode with the cathode → resistor → GND (topology B). Errors have stable codes such as LED_NO_RESISTOR, LED_REVERSED, BUTTON_FLOAT, DHT_NO_POWER and GPIO_SHORT, and warnings cover resistor values, a missing DHT11 pull-up and 5 V power on a 3.3 V board.

```
// Topology A: GPIO -> resistor -> LED anode, cathode -> GND
var rA = findResistorBetween(state, defs, g, anodeNet);
if (rA && cathodeNet === gndNet) { chosenOutput = dp; topology = 'A'; }

if (topology === 'direct')
  err('LED_NO_RESISTOR', 'The LED is connected directly to the GPIO with no
       current-limiting resistor.');

// Button must have one side on GND so the INPUT_PULLUP pin never floats
if (!aGnd && !bGnd) err('BUTTON_FLOAT', 'One side of the button must go to GND ...');
```

7.6 Server-Side Re-validation (Zero Trust)

The browser sends its circuit with every AI request, but the controller never trusts the pin capabilities in it. buildAuthoritativeContext() looks up the board in the backend registry, recomputes the validation result and builds the context that is given to the AI.

```
function buildAuthoritativeContext(rawContext) {
  const board = circuitService.requireBoard(rawContext.board.id);
  const validation = circuitService.validate(circuitInput);
  return {
    board: circuitService.capabilities(board),   // from the server registry
    circuit: { board: boardId, lesson, components, connections },
    validation: validation,                      // recomputed on the server
    code: rawContext.code.slice(0, 8000)         // size-limited
  };
}
```

7.7 AI Guardrails and Provider Abstraction

ai.service.js is the only module that calls the AI provider. A fixed system prompt tells the model to use only the supplied board data, never to contradict the validator and never to claim that code was compiled or tested on hardware. The service builds an Anthropic Messages API request, or an OpenAI-compatible request when configured, so the provider can be changed through environment variables. Prompts differ per feature (explain, generate-code, troubleshoot).

```
const GUARDRAILS = [
  'You are part of the Microcontroller AI Lab, an interactive electronics tutor.',
  'Use ONLY the board capabilities supplied in the context. Never invent pins,',
  'The deterministic circuit validator decides whether a circuit is valid.',
  'Never claim that code was compiled, uploaded, executed or tested on real hardware.',
].join(' ');

function buildBody(system, messages) {
  if (isAnthropic()) return { model: modelName(), max_tokens: maxTokens(),
                              system: system, messages: messages };
  return { model: modelName(), max_tokens: maxTokens(),
           messages: [{ role: 'system', content: system }].concat(messages) };
}
```

7.8 Reliable AI Calls: Timeout, Error Mapping and Usage Recording

Each provider call uses an AbortController so that a slow provider cannot hang the server. Timeouts, rate limits (HTTP 429) and provider errors are mapped to clear application errors, and every call records only metadata – model, feature, token counts, status and duration – never prompts or keys. If no key is configured the AI endpoints return AI_NOT_CONFIGURED while validation, code generation and simulation still work.

```
const controller = new AbortController();
const timer = setTimeout(() => controller.abort(), timeoutMs());
response = await fetch(url, { method: 'POST', headers: buildHeaders(),
                              body: JSON.stringify(buildBody(system, messages)),
                              signal: controller.signal });
if (response.status === 429)
  throw new AppError('AI_RATE_LIMIT', 'The AI provider rate limit was reached...', 429);
usageService.record({ model, feature, inputTokens, outputTokens, status: 'success' });
```

7.9 Code Generation from Resolved Pins

The code generator does not use fixed pin numbers. It reads the pins the validator found in the student's circuit (for example D8 or GPIO2) and converts them with pinNumber(), so D8 becomes 8 on the Uno and GPIO2 becomes 2 on the ESP32. Analog pins A0–A5 map to 14–19. If the circuit has not been validated, the board's default lesson pin is used.

```
function pinNumber(pin) {
  if (/^A[0-5]$/.test(pin)) return 14 + parseInt(pin.slice(1), 10);
  var m = /(\d+)$/.exec(pin);
  return m ? parseInt(m[1], 10) : NaN;
}

code += 'const int LED_PIN = ' + pinNumber(pins.outputPin) + ';  // ' + pins.outputPin +
'\n';
code += 'void setup() {\n  pinMode(LED_PIN, OUTPUT);\n}\n\n';
```

7.10 Browser Simulator as a State Machine

The simulator never runs the generated program. It runs a structured model of the lesson that uses the validated pins. For the LED lesson, setInterval() toggles the LED at the chosen blink interval. For the button lesson, a pressed virtual button is read as LOW because of INPUT_PULLUP and switches the LED on. For DHT11, temperature and humidity sliders produce simulated samples. Start is refused when the circuit is not valid, and unsupported calls found in the code are listed instead of being pretended to run.

```
if (!v || !v.valid) { setStatus('Cannot simulate: fix the validation errors first.');
return; }

timer = setInterval(function () { updateLed(!sim.ledOn); }, sim.intervalMs);

// INPUT_PULLUP: pressed => LOW => LED on
updateLed(pressed);
log('digitalRead(' + sim.inputPin + ') = ' + (pressed ? 'LOW' : 'HIGH'));
```

7.11 Shared State and Event Bus

app.js owns one state object, Lab, that holds the board, lesson, components and connections. Modules do not call each other directly; they communicate through a small publish/subscribe event bus, which keeps canvas, wires, validator, simulator and code editor loosely coupled.

```
Lab.on = function (evt, fn) {
  (listeners[evt] = listeners[evt] || []).push(fn);
};
Lab.emit = function (evt, data) {
  (listeners[evt] || []).forEach(function (fn) { fn(data); });
};
```

7.12 Security and Error Handling

The API key is read from backend/.env on the server only. Request bodies are limited to 256 kb, the question length is limited to 2000 characters, circuits are limited to 200 components and 400 wires, and the request logger never prints bodies or headers. A central error handler returns uniform JSON errors and hides internal details for unexpected failures. The usage endpoints are restricted to localhost by the local-only middleware until authentication is added.

```
function localOnly(req, res, next) {
  if (process.env.USAGE_LOCALHOST_ONLY === 'false') return next();
  const ip = (req.ip || '').replace('::ffff:', '');
  if (['127.0.0.1', '::1', 'localhost', ''].includes(ip)) return next();
  return next(new AppError('FORBIDDEN_LOCAL_ONLY', 'Restricted to local development.',
403));
}
```

8. REST API Endpoints

Method	Path	Purpose
GET	/api/health	Backend status
GET	/api/boards	Configured boards and their capabilities
POST	/api/circuit/validate	Deterministic, board-specific circuit validation
POST	/api/ai/explain	Explain components, circuits and concepts
POST	/api/ai/generate-code	Generate board-compatible code with explanation and libraries
POST	/api/ai/troubleshoot	Diagnose problems from real validation data
GET	/api/usage/status	Usage source configuration (localhost only)
GET	/api/usage/summary	Provider-reported and app-tracked usage, kept separate
GET	/api/usage/details	Per-period usage rows

8.1 Configuration (backend/.env)

Variable	Purpose
AI_PROVIDER, AI_BASE_URL	Selects the provider and request format (Anthropic by default)
AI_API_KEY	Server-side secret; never sent to the browser
AI_MODEL	Model used for all AI features
AI_TIMEOUT_MS, AI_MAX_TOKENS	Request timeout and per-request token cap
AI_USAGE_API_ENABLED, AI_USAGE_API_KEY	Optional provider usage and cost reporting
AI_CREDIT_TOTAL, AI_CREDIT_START	Used to compute the remaining credit balance
PORT, USAGE_LOCALHOST_ONLY	Server port and dashboard access restriction

8.2 Running the Project

```
cd backend
npm install
copy .env.example .env      # add AI_API_KEY
npm start                   # open http://localhost:3000
```

9. Project Screenshots

The following screenshots were captured from the deployed application running in the browser.

9.1 Component Placement

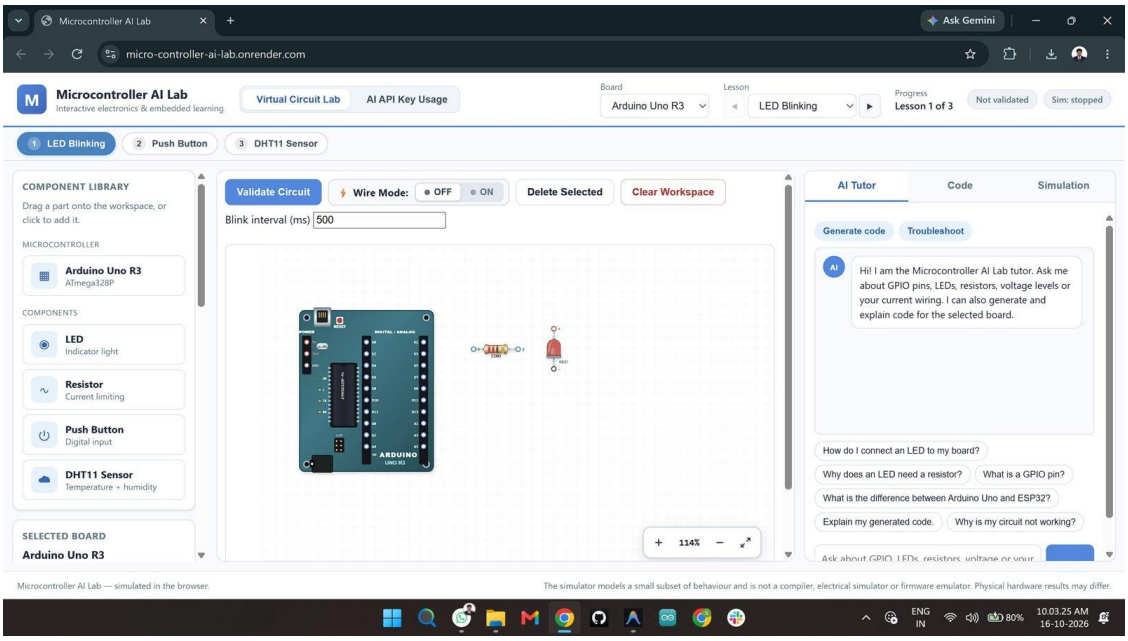


Fig 9.1: Lab workspace – Arduino Uno R3, resistor and LED placed for the LED Blinking lesson (status: Not validated)

9.2 Wiring and AI Tutor

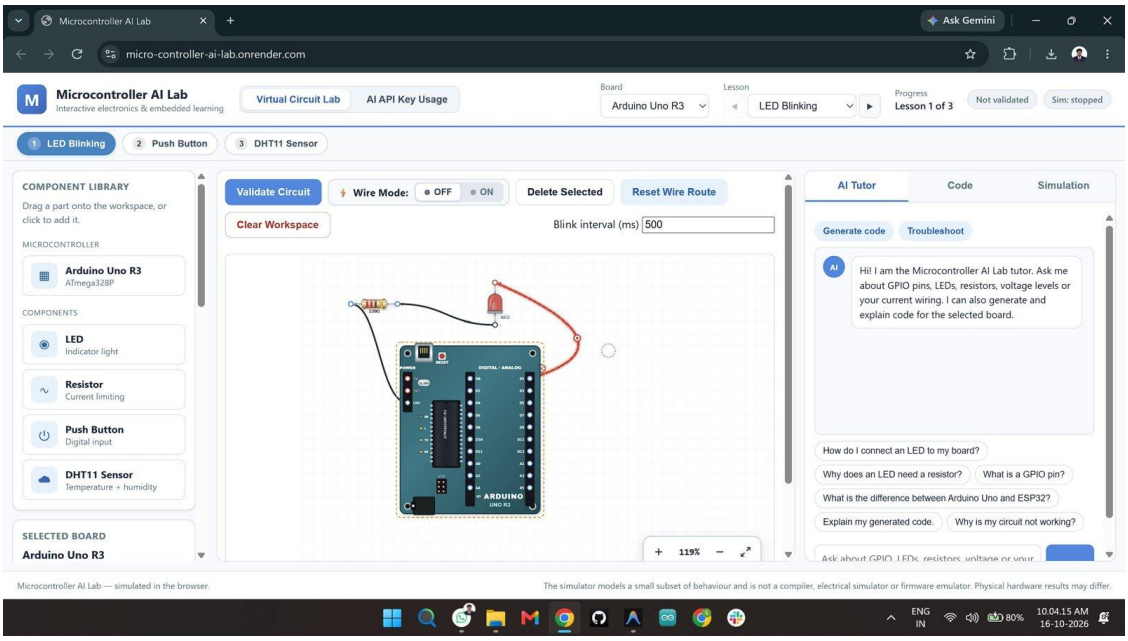


Fig 9.2: Wiring in progress with Wire Mode, and the AI Tutor greeting with suggested questions

9.3 AI Explanation

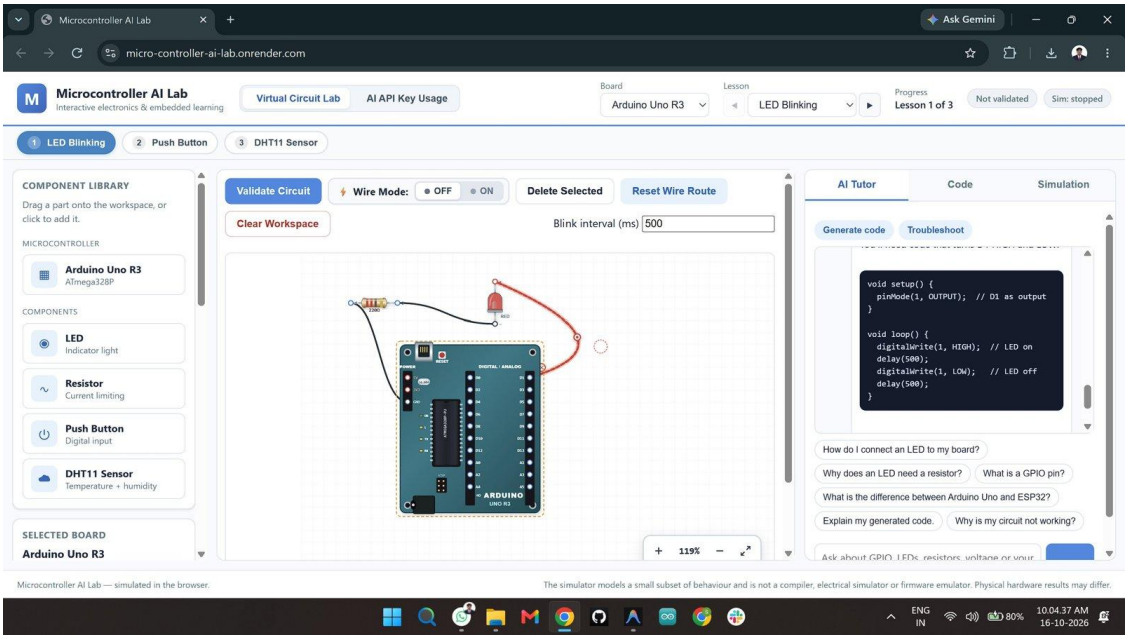


Fig 9.3: Completed circuit and an AI Tutor reply that explains the Arduino code

9.4 Validation and Code Generation

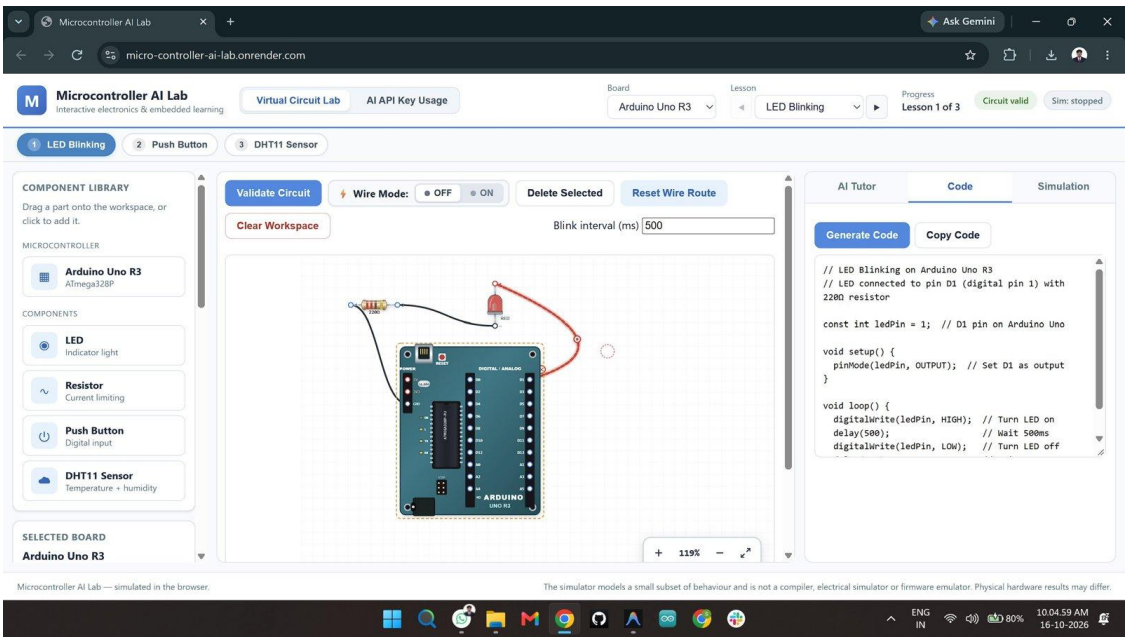


Fig 9.4: Circuit validated and the Code tab showing generated board-compatible code

9.5 Simulation

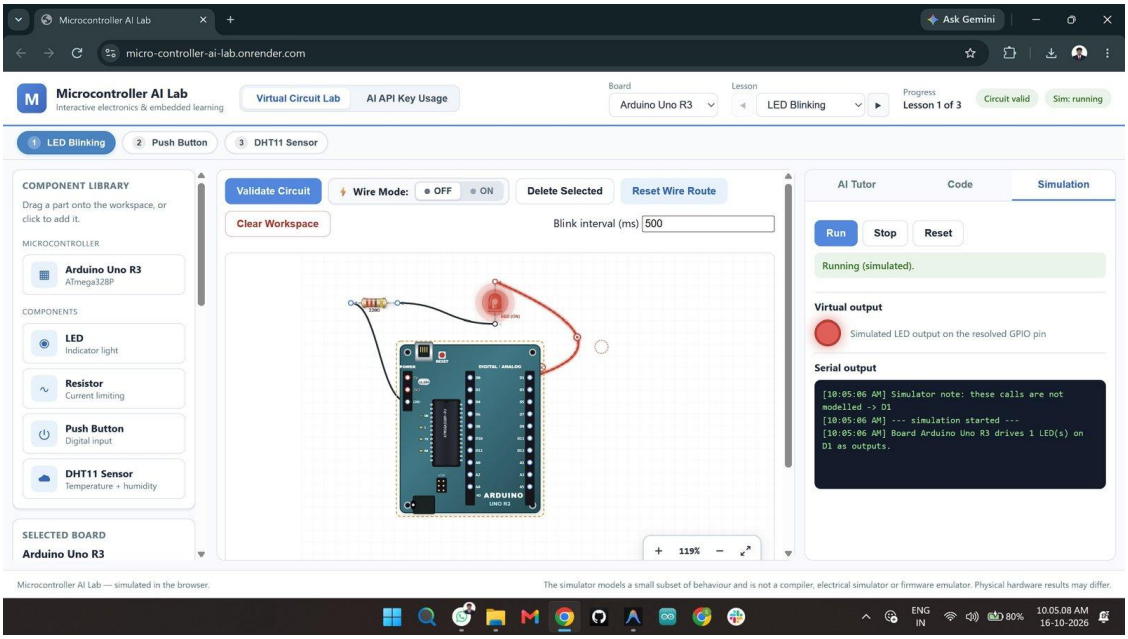


Fig 9.5: Simulation tab – virtual LED lit and serial output ("simulated") while running

9.6 Usage Dashboard

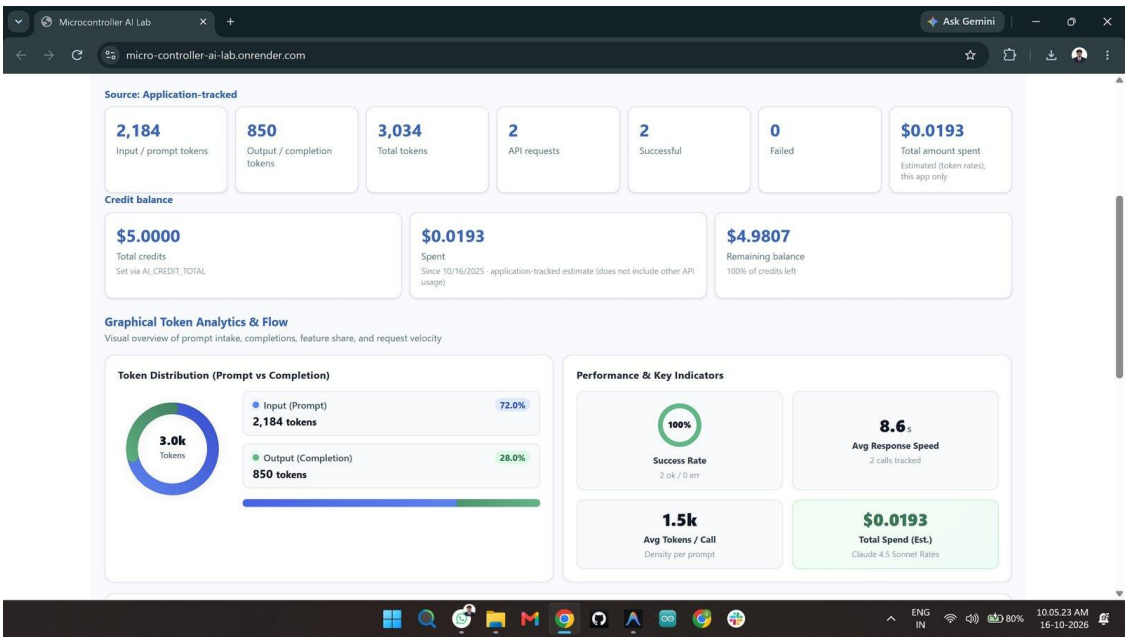


Fig 9.6: AI API Key Usage dashboard – tokens, spend, remaining balance and token distribution

9.7 Usage Details

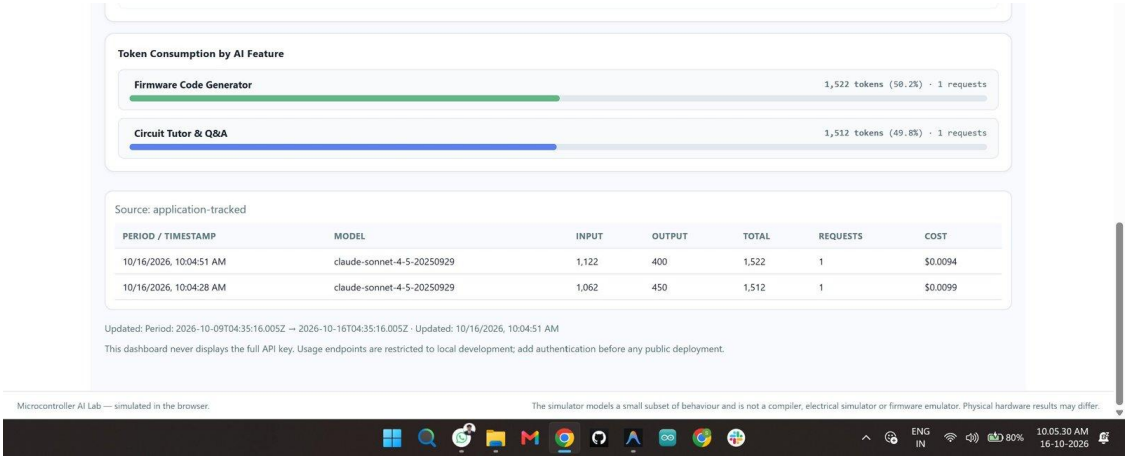


Fig 9.7: Usage dashboard – token consumption by AI feature and per-request details table

10. Result and Discussion

The Microcontroller AI Lab was designed, implemented, deployed and tested successfully. Students can select a board and lesson, build and wire a circuit visually, receive deterministic validation feedback, generate matching code, run a clearly labelled simulation and consult an AI tutor, all in the browser.

10.1 Observations

- A circuit with a resistor, an LED and a ground return was reported as valid and the status badge changed to "Circuit valid" (Fig 9.4).
- The generated code used the pin that was actually wired (LED on D8 with a 220 Ω resistor) and the Run button toggled the virtual LED at the chosen blink interval (Figs 9.4, 9.5).
- The AI tutor answered in the context of the selected board and circuit (Fig 9.3).
- The usage dashboard showed 2 requests, 3,034 total tokens (2,184 input and 850 output), 100 % success and a total spend of about $0.019, with the remaining credit calculated from the configured balance (Figs 9.6, 9.7).

10.2 Test Summary

Feature	Expected behaviour	Result
Board selection	Pins and rules change with the board	Working
Drag-and-drop and wiring	Parts placed, pins connected by wires	Working
Circuit validation	Clear errors/warnings, valid badge	Working
Code generation	Code uses resolved pins for the board	Working
Simulation	LED blink, button input, DHT11 values	Working
AI tutor	Explain / generate / troubleshoot via backend	Working
Usage dashboard	Tokens, cost and balance displayed	Working
Online deployment	Accessible from a web browser	Working

11. Advantages, Limitations and Future Scope

11.1 Advantages

- Safe learning: mistakes cost nothing and no hardware can be damaged.
- Reliable feedback: validation is rule-based, so the AI cannot declare a wrong circuit correct.
- Board-aware: 5 V vs 3.3 V logic, input-only pins and boot-strapping pins are respected.
- Extensible: a new board needs one registry entry; the backend picks it up automatically.
- Secure by design: API keys stay on the server and every request is re-validated.
- Transparent costs: token usage and spend are visible to the instructor.

11.2 Limitations

- It is not a compiler, electrical simulator or firmware emulator; only a small subset of behaviour is simulated.
- Only three lessons and two boards are included, and circuits are not saved between sessions.
- Usage endpoints are limited to localhost and there is no user login yet.
- There are no automated tests; verification was done manually.

11.3 Future Scope

- Add more boards (Raspberry Pi Pico, Arduino Nano) and more lessons (PWM, analog sensors, I2C displays, servo motors).
- Add user accounts, saved circuits and a database for progress tracking.
- Compile code through a safe cloud toolchain and show real serial output.
- Add automated unit tests for the validator and API.
- Add authentication, restricted CORS and rate limiting before wider public use.

12. Conclusion

Microcontroller AI Lab shows how generative AI can be used responsibly in engineering education. By letting deterministic code decide what is correct and the AI decide how to explain it, the platform gives beginners trustworthy feedback while keeping the flexibility of a conversational tutor. The registry-driven design, shared validation modules, server-side re-validation and clear simulation boundaries make the prototype robust and easy to extend, and the deployed version demonstrates that a complete full-stack, AI-assisted learning tool can be built with plain HTML, CSS, JavaScript, Node.js and Express.

13. References

[1] Anthropic API documentation – Messages API – https://docs.anthropic.com

[2] Express.js documentation – https://expressjs.com

[3] Node.js documentation – https://nodejs.org/docs

[4] Arduino documentation (Arduino Uno R3, pinMode, digitalWrite, digitalRead) – https://docs.arduino.cc

[5] Espressif ESP32 Series Datasheet and ESP32 DevKit V1 pinout.

[6] DHT11 Humidity and Temperature Sensor datasheet.

[7] MDN Web Docs – SVG, DOM events and AbortController – https://developer.mozilla.org

[8] Cormen et al., Introduction to Algorithms – Data Structures for Disjoint Sets.